

Development Plan

Squigglr

Team #17, Squigglr
Rayan Nasrallah
Ahmed Elzaria
Muhammad Huzaifah
Shamil Canbolat
Moustafa Moustafa

Table 1: Revision History

Date	Developer(s)	Change
2026-09-26	Muhammad Huzaifah (AI-assisted)	Issue #14: specified workflow, coding conventions, scheduling, meetings, communication, and MIT licensing.

[Put your introductory blurb here. Often the blurb is a brief roadmap of what is contained in the report. —SS]

[Additional information on the development plan can be found in the [lecture slides](#). —SS]

1 Confidential Information?

[State whether your project has confidential information from industry, or not. If there is confidential information, point to the agreement you have in place. —SS]

[For most teams this section will just state that there is no confidential information to protect. —SS]

2 IP to Protect

[State whether there is IP to protect. If there is, point to the agreement. All students who are working on a project that requires an IP agreement are also required to sign the “Intellectual Property Guide Acknowledgement.” —SS]

3 Copyright License

Squiggly adopts the [MIT License](#) for the team’s own work. The complete terms and the copyright notice, “Copyright (c) 2026 Squiggly contributors,” are in the repository’s [LICENSE file](#). MIT permits reuse, modification, distribution, and commercial use, provided the copyright and permission notices accompany copies or substantial portions of the work. The work is supplied without warranty. Contributors must preserve applicable third-party notices when reusing material; the team’s license does not replace the terms of dependencies or other third-party content.

4 Team Meeting Plan

The weekly meetings in Table 2 provide planning, progress checks, and review time. All times use **America/Toronto**, so the local meeting time remains fixed through daylight-saving changes. Virtual meetings use Google Meet; Microsoft Teams holds invitations and reservations. The invitation specifies the format and duration, adjusted to the current deliverables. Physical locations are not published in this plan. Agreed changes or cancellations are recorded in the meeting issue and the invitation, with prompt notification through WhatsApp.

The **Project Manager** chairs meetings and posts an agenda in the relevant GitHub meeting issue at least 24 hours beforehand. Members review the agenda and update their assigned issues before attending. The meeting begins with

Table 2: Recurring meetings

Meeting	Recurrence	Primary purpose
Team planning	Tuesday, 5:00 p.m.	Plan upcoming work, allocate tasks, and review dependencies.
Team status	Saturday, 12:00 p.m.	Review each member’s progress and blockers.
Team review	Sunday, 10:00 a.m.	Finalize and review the week’s work.
Supervisor	Wednesday, 12:30 p.m.	Discuss project updates, deliverable questions, concerns, and feedback.

progress and blockers, proceeds through the agenda and decisions, and ends with the chair confirming action owners and deadlines.

The designated notetaker records attendance, decisions, notes, and action items in the meeting issue’s description within 24 hours after the meeting. Each action has an owner and deadline; substantive work receives a linked task issue. The chair checks that these records capture the agreed actions before the meeting issue is closed.

If the chair or notetaker is unavailable, the team identifies a substitute in the meeting issue before the meeting. Members follow the Team Charter’s absence and emergency policy: notify the team of predictable absences at least 24 hours ahead where possible, and of emergencies as soon as reasonably possible. An absent member reviews the notes and confirms assigned actions within 24 hours of the notes becoming available, unless an emergency prevents this; the team then agrees on a recovery time.

5 Team Communication Plan

Table 3 assigns each communication tool a clear purpose. GitHub is the canonical record: whoever coordinates a consequential conversation records its decision, rationale, and resulting tasks in the relevant issue within 24 hours. Repository deliverables remain version controlled; files shared in Teams supplement that record.

Members acknowledge routine actionable requests within 24 hours. Urgent blockers require acknowledgement within four hours counted during 9:00 a.m.–9:00 p.m. EST. For example, an urgent request at 8:00 p.m. is due for acknowledgement by noon the next day. An acknowledgement provides an answer or an expected response or action time; it does not promise resolution within that window. The sender raises an overdue urgent request directly with the chair for coordination or reassignment. Communicated unavailability and emergencies follow the Team Charter.

Table 3: Communication tools and responsibilities

Tool	Purpose
GitHub	Canonical task records, technical decisions, meeting notes, code and document history, and reviews.
WhatsApp	General conversation, quick coordination, and urgent notifications linking to the relevant GitHub issue.
Microsoft Teams	File sharing and meeting invitations or reservations.
Google Meet	Live virtual meetings.
University email	Formal external correspondence through the Team Liaison. For the instructor and teaching assistants, put the course code in the subject and include the team number and name; contact the instructor by email.

6 Team Member Roles

[You should identify the types of roles you anticipate, like notetaker, leader, meeting chair, reviewer. Assigning specific people to those roles is not necessary at this stage. In a student team the role of the individuals will likely change throughout the year. —SS]

7 Workflow Plan

GitHub connects each task to its implementation, validation, and review. The workflow below applies to code, documentation, and repository maintenance. Authors own their changes through review and closure; reviewers check both the completion criteria and the evidence supplied.

7.1 Git Branches, Commits, and Review

The planned workflow uses short-lived issue branches, `development` for integration, and `main` for stable versions. The `development` branch has not yet been created; current documentation PRs, including this contribution, target `main`. Creating the branch, aligning the README instructions, and configuring its checks and protection are follow-up repository work. Once it is available, use this workflow:

1. Select an assigned GitHub issue with clear completion criteria. Update local `development` from the remote, then create a branch using this pattern:

```
<type>/<issue-number>-<short-description>
```

For example, use `docs/14-workflow-process`. Move the issue to *In progress*.

2. Make focused commits that preserve each contributor's work. Use the commit format below and include the issue reference. Keep unrelated changes in separate issues and branches.
3. Run checks relevant to the change and inspect the results. Push the branch and open a pull request (PR) against **development**. Include the evidence needed to review it and move the issue to *In review*.
4. Address feedback, obtain at least one teammate's approval, and resolve review conversations. All applicable checks must pass. Substantive changes after approval require renewed review before merging.
5. Merge into **development** using **Create a merge commit**. Confirm the issue's completion criteria, update the board, and delete the merged issue branch. Close work completed at this stage manually with a link to its PR; work requiring stable integration remains open.
6. The affected contributors verify their combined changes on **development**, including applicable integration checks and manual user-flow or document review. Fix integration failures through reviewed issue branches before promoting the result.
7. When integration is verified, open a PR from **development** to **main** with the included issues and validation evidence. Obtain at least one approving teammate other than the author, pass all applicable checks, and resolve review conversations. Substantive changes require renewed review. Merge using **Create a merge commit**, verify the stable result, and close the remaining completed issues. Keep the **development** branch for the next integration cycle.

Allowed commit types are **feat**, **fix**, **docs**, **style**, **refactor**, **test**, **build**, **ci**, and **chore**. Use this format:

<type>: <short action describing the change> (#<issue-number>)

<optional explanation of why the change was needed>

The PR title summarizes the ticket and includes its issue number. Its description explains what changed and why, supplies validation evidence (checks run, document review, screenshots, or preview links as applicable), links the relevant issues, and identifies outstanding work. Use **Closes #<issue-number>** only when the entire issue is satisfied; otherwise use **Refs #<issue-number>**. GitHub's automatic issue closure applies when the closing PR merges into the default branch, **main**; close an issue manually if its completion criteria are fully satisfied on **development**. The approving teammate must be someone other than the author. Documentation reviewers use the [Development Plan Checklist](#) and [Writing Checklist](#), as applicable, and inspect the generated PDF.

Merge commits preserve individual commits and attribution. Squash merging and history rewriting on shared branches are prohibited by team policy. To

resolve integration conflicts, coordinate with affected contributors, merge the current target branch into the source branch, resolve conflicts, and rerun relevant checks. This means merging **development** into an issue branch, or **main** into **development** for a promotion PR. Correct mistakes already merged through a reviewed revert or corrective PR.

For each submitted version, the milestone Quality Assurance (QA) coordinator checks the generated PDF and identifies the exact submitted commit with an annotated tag, for example **submission/dev-plan-2026-09-28**. Record the tag and commit in the deliverable’s tracking issue. Preserve existing tags; use a new revision suffix, such as **-r2**, for a corrected submission.

7.2 Issues and Project-Board States

Use the five existing templates as described in Table 4. When none fits, create a blank issue for development, documentation, maintenance, or a bug. Every substantive task records its purpose, relevant context, completion criteria, an accountable assignee, labels, dependencies (or explicitly none), and a target date. Add it to the project board and the applicable milestone. Bug reports also include reproduction steps and expected versus actual behaviour.

Table 4: Existing issue templates

Template	Use
Lecture	Attendance, lecture topics, questions, and relevant notes.
Team Meeting	Team agenda, attendance, decisions, and action items.
Supervisor Meeting	Supervisor or stakeholder agenda, questions, feedback, and actions.
TA Meeting	Teaching Assistant (TA) deliverable feedback, questions, attendance, and follow-up work.
Peer Review	Feedback from another team about a specific artifact.

Apply labels from Table 5 to make work easy to find. When creating an issue, select exactly one priority label alongside the other applicable labels: **High Priority** for urgent tasks, **Medium Priority** for current deliverable tasks, or **Low Priority** for future deliverable tasks. Update that label when the task’s urgency or planned deliverable changes. The **meeting** and **review** labels match the existing templates. Disposition labels accompany an explanation when closing an issue, including a link to the original issue when marking a duplicate.

The assignee maintains the board state using Table 6. A meeting issue reaches *Done* after the meeting occurs and its notes and actions are recorded. Unfinished substantive actions remain in linked task issues, so closing a meeting record does not discard its follow-up work.

Priority labels are the issue’s authoritative priority record. The project’s existing Priority field is optional board metadata, separate from issue creation. When

Table 5: Issue-label categories

Category	Labels
Work classification	bug, fix, enhancement, documentation, repo-maintenance, accessibility
Coordination and feedback	meeting, review, question
Assistance	help wanted, good first issue
Priority (choose one)	High Priority, Medium Priority, Low Priority
Disposition	duplicate, invalid, wontfix

Table 6: Project-board state rules

Column	Entry or completion rule
Backlog	Identified work still needs clarification, prioritization, or scheduling.
Ready	Completion criteria are clear, dependencies are understood, and work is ready to be assigned or started.
In progress	An assignee is actively doing the work.
In review	Reviewable work and validation evidence are available.
Done	Completion criteria are satisfied and required review, merge, or submission is complete.

used, manually keep P0 aligned with **High Priority**, P1 with **Medium Priority**, and P2 with **Low Priority**; no automatic synchronization is configured. Size and Estimate are also optional. Record blockers and their dependencies in the issue and raise them promptly with the Project Manager; an issue remains in its current state with the blocker explained until it can proceed.

7.3 Continuous Integration and Delivery

Continuous Integration (CI) checks proposed changes; Continuous Delivery (CD) prepares a reviewed candidate for an authorized release. Existing automation and planned application checks are distinguished below.

Existing Documentation Automation

The repository's [latex-pages.yml](#) builds affected documentation on relevant PRs targeting **main** and pushes to **main**. Its path filters cover documentation, references, the Makefile, and documentation automation. On pushes to **main**, it updates committed PDFs and the dependency manifest. It also publishes the PDFs through GitHub Pages on non-PR runs, including manual dispatch. Compilation checks syntax; authors and reviewers must still inspect content, links, and layout. Contributors compile PDFs locally for review but do not commit them; the workflow updates **pdfs/** after merging to **main**. Until then, review the local build or the PR's GitHub Actions artifact, rather than expecting the committed PDF to contain the proposed source changes. Before accepting a documentation check, confirm that its log lists the changed document as compiled and that its PDF artifact matches the reviewed source. A successful status alone is insufficient if the run reused a cached PDF without rebuilding the changed document.

Planned Application Checks

When the application scaffold exists, the DevOps Lead will introduce the small pipeline in Table 7, with the Testing Lead coordinating test selection. These checks are planned; the application tooling is not yet installed. Authors run applicable checks before review; GitHub Actions will run them on PRs to **development** and **main**, and after merges to either branch. Committed package scripts and the dependency lockfile will make commands reproducible. Validation jobs use read-only permissions; deployment permissions are granted only to jobs that need them.

Keep documentation compilation separate from application checks. The follow-up implementation must ensure that required checks complete on every PR, even when no relevant files changed. GitHub's [workflow syntax reference](#) explains that skipping an entire workflow through path filters can leave required checks pending. The DevOps Lead will also extend documentation validation to PRs targeting **development**, reconcile direct PDF pushes with mandatory PR protection, and restrict documentation publishing to the intended branch.

Table 7: Planned application validation pipeline

Check	Tool and purpose
Formatting	Prettier check mode detects formatting differences.
Lint	ESLint checks Next.js and TypeScript rules.
Type checking	TypeScript strict checking detects type errors.
Unit/component tests	Vitest and React Testing Library check logic and component behaviour.
Coverage report	<code>@vitest/coverage-v8</code> highlights untested code; reviewers interpret coverage alongside test quality.
Build	A Next.js production build checks application integration.
Browser smoke tests	Playwright checks a complete user flow once one exists; expand checks as flows become available.

Change detection must also fail visibly if its comparison commit is unavailable after a force-push, instead of treating the failure as an empty change list. These are follow-up CI changes, not capabilities claimed for the current workflow.

Preview and Production Release Procedure

The planned Vercel setup provides automatic previews and manual production promotion. Reviewers inspect the preview; a release candidate must build successfully from reviewed `main` after the integration PR from `development` is approved and merged. The milestone QA coordinator verifies applicable automated checks, unresolved defects, and manual checks on target touchscreen and stylus hardware before relevant releases. The Project Manager authorizes promotion, and the DevOps Lead records the deployed commit, verification evidence, and release decision in GitHub. After promotion, the QA coordinator checks the deployed user flow. If verification fails, the DevOps Lead restores the previous working deployment and opens a corrective issue linked to the failed release.

8 Project Decomposition and Scheduling

The [Squiggly repository](#) holds project artifacts and their history. The [Squiggly Project Kanban Board](#) connects issues, ownership, priorities, milestones, and dates to the workflow in Section 7.

8.1 Decomposition and Weekly Planning

Break each deliverable into manageable issues with an owner, completion criteria, dependencies, and a target date. Use parent and sub-issues where they clarify a larger deliverable; identify an integration owner when work spans contributors.

Supporting tasks retain their own deadlines rather than inheriting only the final submission date.

Members update their issues before meetings. Tuesday planning selects work using dependencies, availability, and the Team Charter’s approximate ten-hour contribution per member per week, including scheduled course activities. Account for research, coordination, implementation, and review when allocating work. The Project Manager coordinates reallocation when blockers or uneven workloads threaten the plan.

Use the board’s existing views for distinct purposes:

- **Backlog:** track movement through the work states in Table 6.
- **Priority board:** triage work using the issue’s priority label; if the optional P0/P1/P2 field is populated, keep it aligned manually.
- **Team items / My items:** inspect ownership and individual workload, including review responsibilities.
- **Roadmap:** inspect start and target dates once those fields are populated.

Retain the existing milestone, [Problem Statement](#), [POC Plan](#), [Development Plan](#). The Project Manager will create a milestone for each subsequent major deliverable, including separate milestones for distinct deliverables sharing a date, such as the Software Requirements Specification (SRS) and Hazard Analysis. Future milestones have yet to be created. Link supporting tasks, reviews, and integration work to their applicable milestone.

Aim for review-ready contributions at least **48 hours before each course deadline**, leaving time for integration and corrections. The team must record agreed exceptions before the existing internal deadline passes. The milestone QA coordinator tracks checklist review and unresolved findings; another teammate reviews the coordinator’s own work before submission.

8.2 Course Timeline and Schedule Maintenance

Table 8 follows the [course Google Calendar](#), checked September 26, 2026, using its teaching-week labels and recurrence exceptions. Reading breaks are excluded from week numbering. Dates use America/Toronto; all-day entries do not establish a submission time. The Project Manager checks calendar changes during weekly planning and updates affected issues, milestone plans, and this document. Confirm submission times from course instructions and record them in issues before calculating internal review deadlines.

Proof of Concept (POC) and Verification and Validation (V&V) activities appear alongside document submissions and peer reviews. The proposed custom documents are the *Literature Review Report* for fall and the *Usability Report* for winter. These choices remain proposed until finalized. Demonstration windows are not individual appointments; the team’s assigned slot will be recorded when available. Tentative calendar entries below remain subject to confirmation.

Table 8: Course deliverables and required activities, 2026–2027

Week	Date	Deliverable or required activity
2	Sept. 18, 2026	Team formation, project selection, and approved onboarding.
3	Sept. 22	Biosheets.
3	Sept. 23	Informal Problem Statement/Development Plan review.
3	Sept. 25	Peer-review assignment requests.
4	Sept. 28	Problem Statement and Goals, POC Plan, Development Plan, and Team Charter.
4	Oct. 1	Peer review of initial planning documents.
5	Oct. 7	Informal SRS and Hazard Analysis presentation.
6	Oct. 19	SRS and Hazard Analysis.
6	Oct. 22	SRS peer-review issues.
7	Oct. 28	Informal V&V and POC planning presentation.
8	Nov. 2	V&V Plan.
8	Nov. 5	V&V peer review and POC scheduling requests.
9	Nov. 11	Informal Design Revision –1 / POC discussion.
10	Nov. 16	Design Revision –1 and fall custom document (proposed Literature Review Report).
10	Nov. 19	Design peer review; POC contribution survey and productivity report.
11–12	Nov. 23–Dec. 4	POC demonstration window; individual slot assigned later.
13	Dec. 9	Code-review interviews.
15	Jan. 12, 2027	Informal design-document review.
16	Jan. 18	Design documentation; Rev 0 scheduling requests.
16	Jan. 21	Peer module implementation.
17	Jan. 25	Peer module reflection.
17	Jan. 27	EXPO registration (tentative).
17	Jan. 29	Rev 0 contribution survey and productivity report.
18–19	Feb. 1–11	Scheduled Rev 0 presentation sessions.

Continued on next page

Table 8: Course timeline (continued)

Week	Date	Deliverable or required activity
20	Feb. 23	Code-review interviews.
21	Mar. 1	Final-demo scheduling requests.
21	Mar. 2	Informal V&V Report and winter custom-document review.
22	Mar. 8	V&V Report and winter custom document (proposed Usability Report).
22	Mar. 11	V&V/winter custom-document peer review.
23	Mar. 15–21	Final demonstration sessions.
23	Mar. 18	Rev 1 contribution survey and productivity report.
25	Mar. 31	Final documentation; poster and video deadlines remain tentative.
26	Apr. 6	EXPO (tentative date/time).
26	Apr. 7	Course evaluation (tentative).

9 Proof of Concept Demonstration Plan

What is the main risk, or risks, for the success of your project? What will you demonstrate during your proof of concept demonstration to convince yourself that you will be able to overcome this risk?

10 Expected Technology

[What programming language or languages do you expect to use? What external libraries? What frameworks? What technologies. Are there major components of the implementation that you expect you will implement, despite the existence of libraries that provide the required functionality. For projects with machine learning, will you use pre-trained models, or be training your own model? —SS]

[The implementation decisions can, and likely will, change over the course of the project. The initial documentation should be written in an abstract way; it should be agnostic of the implementation choices, unless the implementation choices are project constraints. However, recording our initial thoughts on implementation helps understand the challenge level and feasibility of a project. It may also help with early identification of areas where project members will need to augment their training. —SS]

Topics to discuss include the following:

- Specific programming language

- Specific libraries
- Pre-trained models
- Specific linter tool (if appropriate)
- Specific unit testing framework
- Investigation of code coverage measuring tools
- Specific plans for Continuous Integration (CI), or an explanation that CI is not being done
- Specific performance measuring tools (like Valgrind), if appropriate
- Tools you will likely be using?

[git, GitHub and GitHub projects should be part of your technology. —SS]

11 Use of AI and LLMs

[This section is a continuation of the previous section. The role of LLMs in your project as important enough to warrant their own section. How will your team be using LLMs for code and documentation development? What tools will you be using? How will you verify the output of your LLMs? —SS]

12 Coding Standard

Squiggler’s Next.js/TypeScript standard uses the official Next.js Core Web Vitals and TypeScript ESLint configurations, with Prettier controlling formatting. This follows the [Next.js ESLint configuration guidance](#) and matches the planned application tools. Application enforcement is planned in Section 7.3; these tools are not yet installed.

12.1 Application Code and Formatting

Use TypeScript for application logic and TSX files for components containing markup, with strict type checking enabled. Use **camelCase** for variables and functions and **PascalCase** for components and types. Choose descriptive names and small, focused functions. Comments explain non-obvious decisions, assumptions, or constraints rather than restating the code. Exceptional use of **any** or a lint suppression requires a documented justification next to the exception and explicit review.

The planned ESLint configuration combines these rule sets:

- `eslint-config-next/core-web-vitals`
- `eslint-config-next/typescript`

Use Prettier defaults for supported application and configuration files, and `eslint-config-prettier` to avoid conflicting formatting rules. Committed scripts will run ESLint, Prettier check mode, and TypeScript strict checking locally and in CI. Reviewers check naming, function scope, comments, and justified exceptions; automated checks cover only the rules enabled in the committed configuration.

12.2 SQL, Python, and Documentation

For authored Structured Query Language (SQL) migrations, use `snake_case` identifiers and uppercase SQL keywords. The Backend Lead and a teammate review schema and access-policy changes for their effect on stored data and permitted access. The existing Python automation helpers follow [PEP 8](#); reviewers check these conventions until dedicated Python linting is introduced.

For $\text{\LaTeX}$, follow the course writing conventions and checklists, including readable source wrapping, descriptive headings, labelled tables, and cross-references. Authors compile changed documents and visually inspect their PDFs for layout, links, and unresolved references before requesting review. The current documentation workflow automates compilation only. ESLint and Prettier do not enforce the project's Python, SQL, or $\text{\LaTeX}$ standards; those conventions require review unless dedicated checks are introduced later.

13 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

[This section is about the use of AI to write this document, not your planned use of AI for your project. You discuss that topic in one of the sections in the body of this report. —SS]

13.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

13.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

13.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. **Why is it important to create a development plan prior to starting the project?**

Ahmed Elzaria:

Creating the development plan helped me recognize that agreeing on a project idea does not automatically mean we agree on how to complete it. A team can share the same goals while having different expectations about responsibilities, deadlines, and acceptable contributions. Discussing these expectations early gives us a common basis for coordinating work and addressing problems before they affect a major deliverable.

While developing the team roles, I initially considered assigning each person a narrow area of implementation. However, I realized that this could restrict members' learning and create dependence on individuals. Making every responsibility shared introduced the opposite problem: accountability became unclear. The approach I settled on combines named leads with shared development duties and explicit task ownership. This showed me that planning involves balancing competing needs, rather than simply dividing a list of tasks.

The technology discussion also demonstrated the value of planning. Clarifying that enhanced drawings should remain editable changed what we needed from the implementation: returning a generated image would not satisfy that expectation. Identifying this distinction early helped expose a feasibility question before investing in an unsuitable approach. I therefore see the development plan as a starting point that makes assumptions visible and guides early investigation, while remaining open to revision as we learn.

Moustafa Moustafa:

A development plan makes it clear who is responsible for what before the work starts. Without one, tasks are unclear and it is easy for someone to say "I don't want to do this" with nothing to point back to. When we

define roles, who works on what, how we split tasks, and which tools each person uses, it saves a lot of friction later.

I have learned this from experience. On a previous project, we did not have a well-defined plan, and it came back to haunt us near the end. All three of us got busy with university just as some milestones were due. Nobody wanted to take them on, but someone had to, and the work ended up being split unevenly across 2 people. With a clear plan, clearly separated tasks, and agreed roles, we could have avoided that situation entirely. That is why I think having the plan in place from the start matters so much for a five-person capstone team.

Rayan Nasrallah:

The benefit I found clearest was being made to sketch the technical implementation at a high level, because doing so exposed what we did not yet know how to build. Committing to a rough shape for the system forced questions that a feature list never raises on its own. Our discussion about keeping refined drawings editable is a good example: the requirement sounds straightforward until you consider the implementation, at which point returning a generated image clearly fails to satisfy it. Identifying that early gives us time to investigate, rather than discovering the problem once work depends on the wrong approach.

These feasibility questions feed directly into scope, which matters more here than it would on an open-ended project. We have two semesters, five members, and a fixed set of deliverables alongside the implementation, so the amount of work a feature requires is as important as how valuable it is. Thinking this through in advance is what let us treat structured practice, adaptive guidance, and progress reporting as core work while moving camera-based tracing and letter activities to stretch goals. Without estimating the effort first, that judgement would have been guesswork.

I also think there is a real difference between discussing these decisions and writing them down. Discussion can produce the feeling of agreement while members leave with different understandings of what was decided, and writing forces the vagueness to surface. The written plan also gives us a concrete reference to revise. I do expect parts of it to prove wrong once implementation begins, since we are planning around technology we have not used yet, so I see its value less in being accurate and more in making our assumptions explicit enough to correct.

Muhammad Huzaifah:

I see the development plan as a way to establish a shared high-level vision before moving into detailed implementation. Agreeing early on responsibilities, workflows, testing, and scheduling helps us understand how our individual contributions fit together and avoids conflicting assumptions. Recording these decisions and their reasons also gives us traceability: when we revise an approach, we can explain what changed and why. For me, the value is in making informed decisions early while keeping them open to

revision as we learn. This gives the team a common starting point without treating the initial plan as fixed.

Our discussions about commit and pull request conventions made this practical for me. We shared the same project goals but had different expectations about how contributions should be recorded and reviewed. Agreeing on these conventions showed me why working expectations need to be explicit early, so that differences do not repeatedly interrupt implementation later.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

Ahmed Elzaria:

I see the main advantage of Continuous Integration (CI) as giving the team consistent feedback whenever changes are proposed. With five members contributing, relying on each person to remember and reproduce every check would make mistakes easier to miss. Automated builds, linting, and tests can identify problems while the relevant change is still small and easier to investigate. For our documentation, an automated L^AT_EX build can also reveal compilation errors before submission.

Continuous Delivery or Deployment (CD) extends this process. Continuous delivery keeps changes ready for release, while continuous deployment automatically releases changes that pass the required checks. For Squiggler, preview deployments would make it easier for teammates to review an interface change and try it on a tablet. However, passing automated checks should not be treated as proof that a change is ready for users. Drawing responsiveness and stylus behaviour still need evaluation on actual devices, and a successful document build does not establish that its content satisfies the rubric.

The main disadvantages are the effort required to configure and maintain the workflows, the time spent investigating failed runs, and the risk of trusting incomplete tests too much. A pipeline can pass while important behaviour remains untested. For example, a scoring function could produce consistently incorrect results if its tests use the same mistaken assumptions as its implementation.

Given our timeline, I favour introducing a small set of useful checks first and expanding them as the application develops. This would allow us to gain early feedback without spending disproportionate effort on automation before the core functionality exists. Human review and device testing would remain part of our process.

Moustafa Moustafa:

I first saw the real value of CI/CD during my previous co-op. It was my first experience on a large project, and I realized that a codebase of that size could not function without it once more than a few people were contributing. The same applies on a smaller scale to capstone. With

five people working in the same repository, we would constantly run into conflicts and broken builds without CI/CD, and it would be hard to tell which change caused a problem. Automated checks on every pull request catch issues early, make it clear which change introduced them, and make it much easier to roll back when something goes wrong.

The main disadvantage is that it can become a bit of a rabbit hole. Some people keep improving their pipeline, adding more optimizations and tools, until maintaining the CI/CD takes more time than the actual work. There is a balance to find: the pipeline should be good enough to let the team work together safely and recover from mistakes, without being over-engineered. For us, that means keeping CI/CD simple and only adding to it when we actually need to.

Rayan Nasrallah:

Continuous Integration means merging our work into the shared branch frequently, with automated checks running on it each time, instead of integrating large amounts of work at once. Continuous Delivery keeps every change that passes those checks ready to release, while continuous deployment releases it without waiting for a person. Our repository already does all of this for documentation: a GitHub Actions workflow determines which documents a change affects, compiles them, commits the resulting PDFs, and publishes them to our project page. Once implementation begins, the same pipeline would run tests and linting rather than only document builds.

The advantage I value most is the effect this has on how we work, rather than what it automates. Because every change arrives as a pull request that has to build before it can merge, work cannot sit unintegrated on a branch for weeks and then arrive all at once near a deadline. Changes stay small enough for somebody other than the author to read properly, and the project advances in increments we can each follow.

The published output stays current for the same reason. Our PDFs always correspond to what is on the main branch, so nobody has to compile everything before a submission and discover a problem then. For the application, preview deployments would let the rest of the team try an interface change on a tablet instead of taking the author's description of it on trust.

I initially thought there were no real disadvantages, and looking more carefully at our own workflow changed that view. The clearest cost is that a passing pipeline is easy to mistake for a correct result. Our problem statement compiled without error while citing references the document did not yet contain, because $\text{\LaTeX}$ has no reason to object to that. The same failure will be available to us in code, where tests can pass while drawing responsiveness on a real tablet remains wrong. Automated checks only cover what we thought to check, so I read a passing run as evidence that nothing obvious broke rather than that the work is right.

Muhammad Huzaifah:

I view CI/CD as a technical investment. Setting up reliable builds, tests, and deployment workflows takes valuable time upfront, and those workflows need ongoing maintenance. However, that effort can accelerate later development by catching bugs early, reducing repetitive manual work, and making releases more consistent. Automated rollback, when configured and tested, can also reduce the impact of a failed deployment. The trade-off is that poor configuration or incomplete tests can give false confidence. I would therefore start with checks that address our most immediate risks and expand them as the project grows, while retaining human review.

Our documentation pipeline made this risk concrete: it reported success while uploading an outdated PDF because change detection failed and compilation was skipped. This showed me that a successful status is only useful if the intended checks actually ran. For future workflows, I would verify both failure handling and the generated output, rather than relying on the status alone.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Ahmed Elzaria:

Our main differences concerned the consequences of missing meetings and the roles and expectations of each team member. These discussions required more communication than simply dividing the deliverable because they established how we would hold one another accountable throughout the project.

For attendance and missed commitments, I wanted expectations that were clear enough to act on while recognizing that illness, emergencies, and unavoidable conflicts can occur. I saw a risk in both extremes: vague consequences could allow repeated problems to continue, while an overly rigid response could treat a genuine difficulty as a lack of effort. Through discussion, we developed an approach with measurable triggers and a staged response. The charter begins with a private conversation and a recovery plan, followed by escalation to the teaching team if commitments remain unmet without an acceptable explanation. This gives the team a basis for taking action while allowing us to understand the circumstances first.

We also differed over how specifically to define individual roles. While working on this section, I found that narrow assignments could make members feel restricted to one component, whereas making everything a shared responsibility left no clear person to coordinate the work. The resulting structure assigns broad lead roles while making software development, testing, documentation, and peer review responsibilities of every member. Assigning Moustafa as the notetaker provides a consistent owner for meeting records, while rotating the quality assurance coordinator each milestone distributes deliverable review duties.

These discussions changed how I understood fairness within the team. Giving everyone the same number of titles does not necessarily create an equal workload, because implementation, coordination, and review require different amounts of effort at different stages. I now see the role assignments as a starting point that must be supported by clear task ownership and regular workload reviews.

My main lesson was that expectations need to be discussed explicitly rather than assumed. Writing down the consequences, role boundaries, and shared duties helped turn our differing views into a practical agreement. However, agreement on the document does not guarantee that the process will work well. We will need to revisit it as the workload increases and check whether the responsibilities and responses remain fair and useful.

Appendix — Team Charter

[borrows from [University of Portland Team Charter](#) —SS]

External Goals

[What are your team’s external goals for this project? These are not the goals related to the functionality or quality of the project. These are the goals on what the team wishes to achieve with the project. Potential goals are to win a prize at the Capstone EXPO, or to have something to talk about in interviews, or to get an A+, etc. —SS]

Attendance

Expectations

[What are your team’s expectations regarding meeting attendance (being on time, leaving early, missing meetings, etc.)? —SS]

Acceptable Excuse

[What constitutes an acceptable excuse for missing a meeting or a deadline? What types of excuses will not be considered acceptable? —SS]

In Case of Emergency

[What process will team members follow if they have an emergency and cannot attend a team meeting or complete their individual work promised for a team deliverable? —SS]

Accountability and Teamwork

Quality

[What are your team’s expectations regarding the quality of team members’ preparation for team meetings and the quality of the deliverables that members bring to the team? —SS]

Attitude

[What are your team’s expectations regarding team members’ ideas, interactions with the team, cooperation, attitudes, and anything else regarding team member contributions? Do you want to introduce a code of conduct? Do you want a conflict resolution plan? Can adopt existing codes of conduct. —SS]

Stay on Track

[What methods will be used to keep the team on track? How will your team ensure that members contribute as expected to the team and that the team performs as expected? How will your team reward members who do well and manage members whose performance is below expectations? What are the consequences for someone not contributing their fair share? —SS]

[You may wish to use the project management metrics collected for the TA and instructor for this. —SS]

[You can set target metrics for attendance, commits, etc. What are the consequences if someone doesn't hit their targets? Do they need to bring the coffee to the next team meeting? Does the team need to make an appointment with their TA, or the instructor? Are there incentives for reaching targets early? —SS]

Team Building

[How will you build team cohesion (fun time, group rituals, etc.)? —SS]

Decision Making

[How will you make decisions in your group? Consensus? Vote? How will you handle disagreements? —SS]